

TGweaver: Synthesizing Transaction Graphs for De-anonymization Analysis

Fajie Wu
Sun Yat-sen University
Zhuhai, China
wufj7@mail2.sysu.edu.cn

Jiajing Wu*
Shenzhen Research Institute of
Sun Yat-sen University
Shenzhen, China
wujiating@mail.sysu.edu.cn

Zhiying Wu
Sun Yat-sen University
Guangzhou, China
wuzhy95@mail2.sysu.edu.cn

Jun Chen
Ant Group
Shanghai, China
retsina.cj@antgroup.com

Tao Wang
Sun Yat-sen University
Zhuhai, China
wangt533@mail2.sysu.edu.cn

Longjian He
Sun Yat-sen University
Guangzhou, China
helj33@mail2.sysu.edu.cn

Bowen Song*
Ant Group
Hangzhou, China
bowen.sbw@antgroup.com

Weiqliang Wang
Ant Group
Hangzhou, China
weiqliang.wmq@antgroup.com

Abstract

Mixing services run on blockchain trading systems, enhancing the transaction privacy of blockchain users. Yet, in recent years, the mixing services provide fertile ground for concealing illicit fund flows. Therefore, security experts make great efforts to find an effective de-anonymization of mixing services. Unfortunately, current de-anonymization technologies are constrained by a fundamental issue, i.e., the lack of a comprehensive, extensive, and reliably labeled benchmark dataset. To address this problem, we propose a new method for acquiring mixing transaction data. We design and implement a method named TGweaver, which actively executes the complete mixing workflow within a simulated blockchain environment. Furthermore, to enhance the realism of the dataset, we introduce a “behavioral fingerprint” mapping strategy. Ultimately, the proposed dataset includes over 891K transactions, scaling existing benchmark sizes by 2 to 4 orders of magnitude. In experiments, we use the proposed data to systematically evaluate existing de-anonymization techniques. Experimental results reveal that the current mixing address linking methods, based on heuristic rules, lacks generalization capability in complex scenarios, exhibiting low precision. In contrast, the methods utilizing supervised learning demonstrate significant advantages.

CCS Concepts

• **Security and privacy** → *Network security*.

Keywords

Mixing Service, De-anonymization, Blockchain

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792318>

ACM Reference Format:

Fajie Wu, Jiajing Wu, Zhiying Wu, Jun Chen, Tao Wang, Longjian He, Bowen Song, and Weiqliang Wang. 2026. TGweaver: Synthesizing Transaction Graphs for De-anonymization Analysis. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792318>

1 Introduction

Blockchains, such as Bitcoin [27] and Ethereum [12], are peer-to-peer transaction systems. The mixing services [11, 26, 34, 35] are smart contracts that run on the blockchain and provide the functionality to break the link between trading addresses. By leveraging these services, users can conceal the genuine source and destination of funds to achieve the goal of privacy enhancement. Consequently, mixing services have inevitably been exploited by malicious users [40], becoming the tools in money laundering, terrorist financing, and the movement of illicit proceeds following cyber attacks [9, 14, 33].

To break the anonymity provided by mixing services, some studies [8, 13, 22–24, 32, 42] have endeavored to analyze account behavioral characteristics. However, current methodologies exhibit substantial deficiencies in practical deployment. Heuristic rules are heavily contingent upon expert prior knowledge [20, 37, 41]. Paradigms based on learning are constrained by the scarcity of ground truth labels. Supervised models often fail to generalize to novel patterns of behavior, while unsupervised models lack rigorous validation on benchmarks of an extensive scale. Although numerous attempts have been made in both academia and industry, these efforts collectively highlight and are constrained by a more fundamental issue, namely the acute scarcity of datasets containing accurate labels for mixing transactions on an extensive scale. However, constructing such a dataset is challenging: 1) **Lack of ground-truth labels**: Current mixing transaction samples come from expert reports or community comments, whose accuracy cannot be guaranteed and may even contain errors. 2) **Cost limitations**: Acquiring mixing transaction samples by participating

in mainnet mixing services requires substantial transaction fees, which restricts the scale of datasets that can be built.

To fundamentally address the challenges mentioned above, this paper proposes a novel methodology for acquiring mixing transaction data. Specifically, we execute the complete mixing services workflow within a forked mainnet environment, thereby generating an extensive-scale mixing transaction data without error labels. Our core contributions can be summarized as follows:

- (1) Mixing transaction synthesis framework. We designed a framework that constructs a comprehensive dataset by simulating user interactions with mixing contracts.
- (2) A dataset without error labels. We generated a novel synthetic dataset comprising 100K wallets and over 891K mixing transactions, scaling existing benchmark sizes by 2 to 4 orders of magnitude.
- (3) Systematic evaluation of existing techniques. We benchmarked the reliability and applicability of mixing address linking methods. The results demonstrate that supervised learning approaches generally outperform unsupervised methods; however, even the state-of-the-art method, MixBroker [15], achieves a HR@1 of only 1.89% when evaluated against an anonymity set size of 1000. This underscores that existing tools still find it challenging to deliver accurate tracing results.

The source code for this study is available at the provided address: <https://github.com/Riemannsgood/TGweaver>.

2 Preliminaries

This section begins with an introduction to the relevant background, followed by a systematic survey of existing research.

2.1 Background

2.1.1 Blockchain Foundations. The foundations of blockchain infrastructure involve several critical components. **Remote procedure call, or RPC**, is a communication protocol enabling external applications to request tasks such as status queries from blockchain nodes without managing underlying network details. **Mainnet** represents the production environment of a blockchain protocol and the public network acknowledged by the vast majority of users. A **full node** serves as a server that maintains the entire ledger state and ensures the compliance of every transaction by verifying all blocks. **Forking** is the technology of creating snapshots of the state of the mainnet to reconstruct an independent chain. Finally, a **blockchain Wallet** acts as a tool for managing private keys and authorizing transactions through digital signatures to interact with the blockchain.

2.1.2 Mixing and Privacy. The mechanisms of mixing and privacy rely on specific cryptographic protocols. **Zero knowledge proofs, or ZKP**, refer to a cryptographic method allowing one party to prove the validity of a statement without revealing sensitive information beyond the statement itself. **Mixing services** represent a category of protocols for privacy enhancement where their core function is to mask the flow of funds and obscure the original source of assets by aggregating capital from a large number of different participants. **Tornado.cash** is a decentralized mixing

service on Ethereum utilizing zero knowledge proofs to provide robust privacy protection. **De-anonymization** is a data analysis technique utilized to reidentify individuals or link independent accounts within anonymous datasets; within the blockchain domain, this process entails the analysis of transaction records on public ledgers to compromise the anonymity of users. The mixing process begins with a **deposit**, the initial operation of transferring specific denominations of assets into mixing services contract addresses while generating cryptographic secrets, and concludes with a **withdrawal**, the anonymous extraction of assets to new destinations secured by proof files that hide the deposit origin. The **recipient** is the terminal address that ultimately receives the funds in a withdrawal transaction. Under the EIP-1559 protocol, the **gas price** represents the cost per unit of gas, consisting of *baseFeePerGas* as the minimum network rate, *maxPriorityFeePerGas* as validator tips, and *maxFeePerGas* as the absolute upper limit for total costs. Furthermore, the **Ethereum name service, or ENS**, is a distributed naming system that maps human-readable names to blockchain addresses through smart contracts. A **router** is a specialized smart contract that manages and directs the flow of assets between different liquidity pools or decentralized protocols, while a **relayer** acts as a third party agent that submits transactions on behalf of users to prevent identifiable links between withdrawal accounts and the source of gas fees. Finally, **deFi** refers to financial protocols built on public blockchains that provide services such as lending or trading without centralized intermediaries.

2.2 Related Work

Research concerning mixing services mainly focused on two primary objectives: mixing services detection and mixing address linking. The former entails the detection of transactions or addresses associated with mixing contract, whereas the latter seeks to establish linkages between deposit and recipient addresses.

2.2.1 Mixing Services Detection. Some research on mixing focused on transaction structural and standardized denominations. For instance, Wu [40] proposed a universal model utilizing seed inputs to discover outputs. Prado-Romero [30] modeled users and transactions as a social network to identify mixing addresses through anomaly detection within communities. To enhance generalization, the research paradigm increasingly pivoted toward representation learning. Ardeshtir [36] applied pattern recognition to classify mixing services and identify associated addresses. Similarly, Xu [18] introduced a framework of ensemble learning with two layers using transaction subgraphs to characterize address behaviors. Wu [40] developed a generalized abstract model to detect the mixing services.

2.2.2 Mixing Address Linking. Linking mixing addresses serves as the key task within the scope of de-anonymization. Current methodologies can be divided into heuristic rules and representation learning.

Heuristic Rules: Such approaches depend on associating private information exposed by the operational lapses of users. This information can be classified into high certainty, which possesses a significant probability of accurately linking mixing addresses, and

behavior patterns, which describe transaction habits. Details are provided in Table 1.

Representation Learning: Some techniques leverage supervised learning or unsupervised learning methods. These approaches build upon foundational methods such as random walks like DeepWalk [29] and node2vec [17], attention mechanisms like GAT [25], and sequence modeling like LSTM [19]. Du [15] proposed MixBroker to construct user interaction graphs and use graph neural networks to learn node embeddings. Wang [39] introduced MixLinker to improve accuracy through tensor computation and modeling of asymmetric information. Hong [20] utilized static and dynamic parameters of mixing services to identify links between inputs and outputs. Garcia-Alfaro [16] applied address tainting to trace transactions. Hu [21] developed BERT4ETH to model patterns of transaction sequences through the Transformer architecture. Ferenc Béres [10] proposed graph representation learning on user interaction graphs to mine behavior information.

2.2.3 Existing Datasets. Apart from the research mentioned above, the acquisition of high quality datasets has stagnated. Current efforts to collect data rely on two primary strategies.

The first strategy is based on transaction features and behavior patterns. Du [15] acquired associated transactions by linking addresses that share the same ENS records. Hong [20] developed a recursive algorithm using parameters such as fixed fees and maximum delays of the Helix [7] service, which represents a historical Bitcoin mixing platform.

The second strategy utilizes public intelligence. Wang [37] integrated incident databases along with information from platforms such as Etherscan [1], SlowMist [6], and Rekt News [5]. By comparing hacker addresses with deposit and withdrawal records in mixing contracts, they extracted transaction data involving illicit fund flows. Although reports from security experts or community comments provide convenient sources, the reliability of this information remains insufficient.

3 Method

To overcome prevalent issues in existing datasets, such as low label credibility, incomplete coverage of behavioral patterns, and restricted scale, we propose a comprehensive methodology for dataset construction. Specifically, the proposed approach aims at generating transactions about mixing services, rather than extracting from anonymous data already residing on the mainnet.

3.1 Overview

Intuitively, we can leverage blockchain forking to construct a controlled environment and participate in mixing services within this environment. Such an approach enables the deterministic linkage of all newly generated mixing transactions. In this way, we propose TGweaver, a data construction model, which primarily comprises two stages as illustrated in Figure 1. Furthermore, we categorize the features within the dataset into two primary classes, specifically behavioral fingerprints and mixing transaction features.

3.1.1 Stage 1: Mixing Transaction Features Generation. The core objective of this stage is to generate mixing transaction data. Mostly, it involves three steps:

Environment Setup: Utilize forking tools to fork the Ethereum mainnet at a specific block height and establish a dedicated test chain.

Transaction Execution: The transaction execution unit executes the complete transaction workflow, which encompasses account funding, depositing into the mixing contract addresses, waiting for a designated interval, and initiating the withdrawal. During this procedure, the distribution control unit modulates the feature selection preferences governing the transaction execution.

Label Generation: During the execution of the mixing transactions, the transaction execution unit records the correspondence between each withdrawal and its original deposit addresses.

3.1.2 Stage 2: Behavioral Fingerprint Mapping. This stage primarily serves to enhance the richness and validity of the dataset and comprises two main steps:

Extraction: All transaction addresses interacting with the mixing contracts are retrieved, and their complete transaction records on the mainnet are collected to constitute the behavioral fingerprints.

Mapping: The mapping control unit establishes the mapping between these behavioral fingerprints and the mixing addresses on the test chain according to predefined logic.

3.2 Key Features

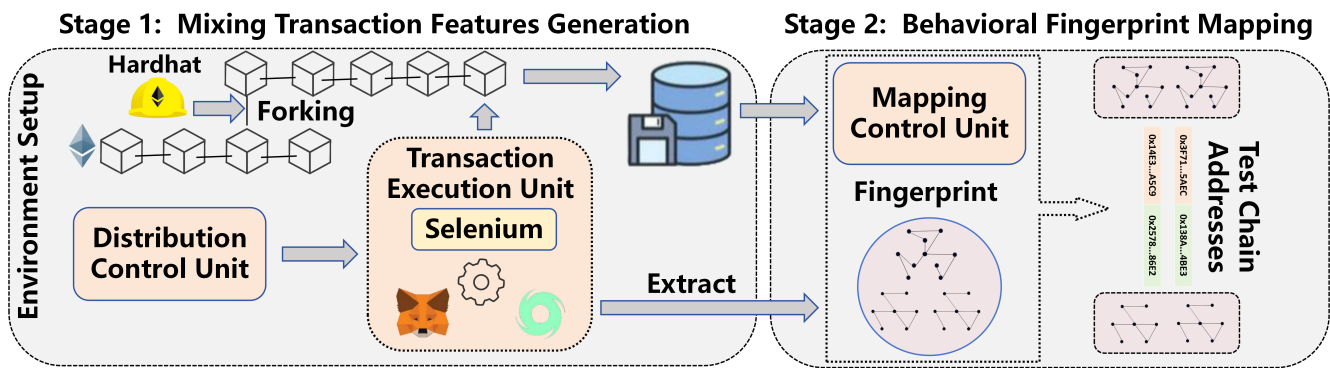
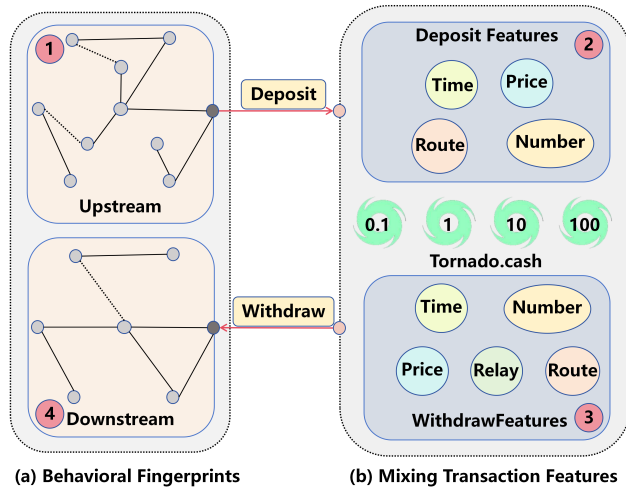
To systematically construct and clearly delineate our methodology, we categorize the features observed throughout the fund flow lifecycle, which covers the complete process of funds from their origin to the final transaction as illustrated in Figure 2, of mixing services into two primary classes of mixing transaction features and behavioral fingerprints.

3.2.1 Mixing Transaction Features. Figure 2b presents the mixing transaction features designed to characterize the intrinsic attributes of deposit and withdrawal operations. These features constitute the core dimensions of interaction with mixing contracts. Because these attributes directly reflect the behavioral preferences of users, they represent the most frequently analyzed data in mixing address linking research [15, 21]. Detailed definitions of these features are as follows:

- **Time:** Refers to the time zone associated with the block timestamp of the transaction, used to capture the user's temporal behavioral preferences.
- **Price:** Encompasses attributes related to the Ethereum Gas mechanism, specifically including *maxPriorityFeePerGas* and *maxFeePerGas*.
- **Route:** Describes the choice between utilizing a router contract for transaction execution and interacting directly with the mixing contract address.
- **Relay:** Users must choose between paying a service fee to a relay to initiate the withdrawal on their behalf or utilizing their own private address.
- **Number:** The total count of deposit or withdrawal transactions initiated by a specific address, used to measure user activity levels.
- **Amount:** The fixed denomination of the transaction.

Table 1: Taxonomy of Heuristic Rules for Address Linking in Mixing Services

Category	Rule	Explanation
High Certainty	Same address reuse [37]	A single address performs both deposit and withdrawal
	Direct transaction [10]	Transfer between two addresses using the same mixing services
	Unique gas price [10]	Sharing identical and infrequent gas fee parameters, including <i>maxPriorityFeePerGas</i> and <i>maxFeePerGas</i>
Behavioral Pattern	Short time gap [37]	Deposits and withdrawals occurring within a very brief interval
	Denomination consistency [41]	Matching counts of transactions across multiple pools
	Mining credit [41]	Linking via deposit durations derived from reward points
	Timezone alignment [10]	Similarity in the active hours of two addresses

**Figure 1: Architectural overview of the TGweaver framework****Figure 2: A holistic characterization of the complete mixing lifecycle**

3.2.2 Behavioral Fingerprints. Behavioral fingerprints are designed to characterize the on-chain interaction trajectories and the operational habits of mixing service participants beyond the

mixing process. Prior research exemplified by MixLinker [39] has demonstrated that behavioral fingerprints effectively enhance the performance of methodologies for linking addresses within mixing services. As illustrated in Figure 2, the fund flows exhibit four distinct stages, throughout the complete mixing lifecycle: the upstream phase, the deposit phase, the withdrawal phase, and the downstream phase. Users typically participate in DeFi activities or perform transfers in the upstream phase. This is followed by the deposit phase, where assets are committed to the mixing contract for privacy enhancement. The withdrawal phase enables users to reclaim funds to new addresses. Finally, users initiate new on-chain operations in the downstream phase. Fingerprints encompass the complete set of transaction data within both the upstream and downstream phases.

3.3 Mixing Transaction Features Generation

3.3.1 Environment Setting. This step constructs a simulation environment similar to the mainnet through a series of specific configurations. Considering that the Tornado.cash contract on the Ethereum mainnet handles the most substantial volume of mixing services fund flows, our study primarily focuses on the Ethereum mainnet and the relevant Tornado.cash smart contracts.

1) Blockchain Environment Settings: TGweaver establishes an independent test chain by forking the Ethereum mainnet for

Table 2: Forking Parameter Settings

Parameter	Settings
Block Height	$Blockheight_1 = 21525891$, $Blockheight_2 = 22080568$
Block Intervals	Selected from [12, 24, 36] seconds based on frequency [99.312%, 0.682%, 0.006%]
$baseFeePerGas$	Maintain within EIP-1559 bounds

simulation. On the one hand, as the forked test chain replicates the entire state of the mainnet at a specific block height, encompassing account balances, contract states, historical transactions, and consensus rules. This ensures that the transaction execution environment on the test chain is consistent with the live mainnet. On the other hand, compared to public test networks such as Goerli or Sepolia, which serve as standard public platforms where developers test applications before they go live on the mainnet, an independent forked environment enables the acquisition of unlimited tokens, the impersonation of any account, and the manipulation of timestamps, thereby significantly enhancing the feasibility of complex experimental scenarios.

2) Parameter Settings: The parameters of the test chain were configured as specified in Table 2. The block height values of 21525891 and 22080568 denote the initial forking point and the cut-off point for transaction execution respectively. These heights correspond to the dates of January 1, 2025, and March 19, 2025. Based on Ethereum mainnet statistics, block intervals of 12, 24, and 36 seconds are assigned probabilities of 99.312%, 0.682%, and 0.006%. The computation of the $baseFeePerGas$ follows the EIP-1559¹ protocol, which is the current execution protocol of the Ethereum mainnet. This standard defines the specific calculation methodology for the base fee. Notably, the $baseFeePerGas$ is dynamically adjusted based on block utilization. Since the transaction frequency in a test chain is significantly lower than that of the mainnet, the $baseFeePerGas$ would rapidly decay toward its minimum value due to insufficient block load without active intervention. To ensure the dynamic consistency of the test chain, we injected a proportional volume of background transactions based on mainnet transaction densities during the corresponding period between block $Blockheight_1$ and $Blockheight_2$, thereby stabilizing the $baseFeePerGas$ within a fluctuation range consistent with the mainnet.

3) Tools Settings: The selection of tools is enumerated in Table 3. Hardhat was selected as the tool for forking because it provides faster wallet responses compared to alternatives like Ganache [38], Foundry [2], Nethermind [4], and Lighthouse [3], all of which represent common tools for blockchain forking. MetaMask is prevalent blockchain wallet extension utilized for managing and signing transactions. The official website interface of Tornado.cash [31] was chosen as the interaction tool for transaction submission because the web platform is the most popular method for using mixing services. Moreover, this web includes all functions necessary to select specific transaction features.

Table 3: Selected Tools for Mixing Transaction Feature Generation

Category	Tool
Forking	Hardhat
Wallet	MetaMask
Interface	Tornado.cash official website [31]

3.3.2 Transaction Execution. This stage executes the mixing transaction workflow within the forked environment to generate raw transaction records for the dataset.

1) Initialization Procedure:

- a. Forking:** A full node is deployed through an Ethereum full node project². Subsequently, an independent test chain is instantiated by forking the Ethereum mainnet at block height of $Blockheight_1$ utilizing the Hardhat.
- b. Interface Connection:** The test chain RPC interface is integrated into both the MetaMask wallet and the Tornado.cash official web to establish communication across different components.
- c. Account Initialization:** Mixing transaction addresses are instantiated within the test chain. To ensure uninterrupted execution, we quantify the capital requirements for each address beforehand and inject the necessary funds, thereby guaranteeing adequate funds for the entire transaction sequence.

2) Transaction Preference Selections: To ensure mixing transactions on test chain exhibit high realism, distribution control unit adjust the selection of mixing transaction features to match the distribution observed within Ethereum mainnet. The specific workflow is detailed as follows. First, a systematic quantification is performed on various mixing transaction features observed on the mainnet. Subsequently, unsupervised clustering algorithms are employed to recognize latent patterns within these statistical distributions and establish interval partitions. Finally, the distribution control unit generates feature preferences based on these probability distributions and transfers them to the transaction execution unit to be implemented during the transaction generation process. Note that we leverage the Jenks natural breaks optimization [28] as the clustering algorithm. For one thing, this algorithm seeks optimal break points along a numerical axis to form intervals that do not overlap, which aligns with the requirements of this study. For another, the Jenks natural breaks optimization provides statistical advantages by minimizing variance within intervals and maximizing variance between intervals, thereby ensuring that the partitions exhibit high heterogeneity in a statistical sense. Moreover, this algorithm effectively handles data with non-uniform distributions and accurately identifies critical points where data density changes.

To facilitate understanding, the following demonstration utilizes $maxPriorityFeePerGas$ as an illustrative example. First, we analyzed the frequency distribution of $maxPriorityFeePerGas$ within the four core Tornado.cash contracts representing denominations of 0.1, 1, 10, and 100 ETH on the Ethereum mainnet. To prevent

¹<https://github.com/ethereum/EIPs/blob/master/EIPS/eip-1559.md>

²<https://github.com/ethereum/go-ethereum/>

high frequency values from biasing the interval partitioning during clustering, we extracted several of the most frequent values. These values were treated as independent discrete candidates. Subsequently, the Jenks algorithm was applied to the remaining values to determine optimal boundaries, resulting in a bifurcated probability distribution composed of both high frequency discrete values and continuous intervals, as illustrated in Figure 3.

3.3.3 Label Generation. This step represents the most critical component of the data generation process. Since the transaction execution unit initiates all deposit and withdrawal transactions, the addresses of both transactions are entirely visible to this unit. Consequently, the module can deterministically establish the linkage between corresponding deposit and withdrawal transactions, thereby generating ground-truth labels without errors.

3.4 Behavioral Fingerprint Mapping

3.4.1 Extraction. We systematically retrieved all mixing transaction addresses from the four primary Tornado.cash contracts and extracted exhaustive historical transaction records for each unique address, covering all blocks from the contract deployment to *Blockheight*₁, to construct behavioral fingerprints.

3.4.2 Mapping. To enhance dataset richness while maintaining logical rigor, behavioral fingerprints from the Ethereum mainnet are mapped to mixing addresses on the test chain according to Equation 1:

$$S = \bigcup_{f \in \mathcal{F}} \left(\bigcup_{j=1}^{J_f} \bigcup_{k=1}^{K_f} \{(P_j^f, A_k^f)\} \right). \quad (1)$$

Here, S represents the resultant dataset after the completion of the mapping procedure. $\mathcal{F}$ denotes the complete space of feature combinations constructed via the Cartesian product of mixing features delineated in Section 3.2.1. The seven dimensions of mixing features encompass 67 discrete preference values and intervals, as detailed in Table 9. This combinatorial process yields a total of 119,808 unique feature preference groups, which together constitute the complete feature space $\mathcal{F}$. The vector $f \in \mathcal{F}$ represents a specific feature configuration. P_j^f and A_k^f signify the j th fingerprint and the k th mixing address, respectively, both of which possess an identical feature vector f . The core logic of the mapping process dictates that a fingerprint and a mixing address are grouped and exhaustively paired only when their feature configurations are perfectly matched, as illustrated in Figure 4. Furthermore, J_f and K_f represent the cardinality of the behavioral fingerprint set and the mixing address set, respectively, under the specific feature combination f . This design aims to overcome the limitations of label sparsity on the Ethereum mainnet and expand the mixing lifecycle by simulating potential mappings between fingerprints and features. The mapped test chain addresses inherit the complete transaction history of the corresponding fingerprints from the mainnet. Since the mapping process is strictly conducted within the same feature preference groups, the capital flow patterns represented by the fingerprints are inherently aligned with the mixing requirements on the test chain, eliminating potential issues such as capital shortage or behavioral logic breaks within the synthesized transaction data.

Table 4: Experimental Configuration

Category	Configuration
Hardware	CPU: 2 × Intel Xeon Gold 6148 (20 cores) RAM: 128GB DDR4 (3200MHz)
Software	OS: Ubuntu 20.04.6 LTS Languages: Python 3.9, Solidity 0.8.17 Framework: Selenium & ChromeDriver
Libraries	Data Processing: Pandas 2.1.0, NumPy 1.26.0 ML Frameworks: PyTorch 2.1.0, Scikit-learn 1.3.0

4 Experiment

In this section, we focus on two objectives: validating the effectiveness of the synthesized dataset and evaluating the performance of current mixing address linking methods.

4.1 Experimental Configuration

To ensure the accuracy and reproducibility of the results, all experiments were conducted in a uniform environment. Detailed configuration parameters are provided in Table 4.

Dataset: The datasets employed in these experiments are summarized in Table 5. *Dataset_S* denotes the synthetic dataset generated by the TGweaver framework. In contrast, *Dataset_{ENS}* comprises mainnet mixing transaction data identified through ENS information. *Dataset_{TC}* encompasses all transactions collected from Tornado.cash on the mainnet, ranging from its contract deployment to block height *Blockheight*₁. To ensure a rigorous comparison, *Dataset_{S1}* was constructed by randomly sampling from *Dataset_S* to match the volume of *Dataset_{ENS}*. *Dataset_{S2}* represents a targeted subset of *Dataset_S*, specifically filtered to align with the behavioral characteristics of recipient addresses transaction number observed in *Dataset_{ENS}*. *Dataset_{S3}* was established by randomly sampling from *Dataset_S* to match the volume of *Dataset_{TC}*.

Evaluation Metrics:

- Classification performance: Accuracy, Precision, Recall, F1, FPR, FNR, AUC.
- Mean Average Rank, (MAR): the average rank of all associated addresses within the anonymity set. Anonymity set: the complete pool of deposit addresses that could potentially match the recipient of a given withdrawal transaction.
- Hit Ratio @k, (HR@k): the percentage of total evaluation cases where the target address is ranked among the top K candidates. Candidates: the anonymity set after its search scope has been narrowed.

4.2 Validity Verification

4.2.1 Experimental Design. We established a validity verification scheme to evaluate the practical utility of the synthetic dataset within the context of mixing address linking research. The fundamental rationale of this approach is that the synthetic dataset is considered effective and reliable if a model trained on the synthetic dataset reaches a level of performance on the Ethereum mainnet dataset that closely mirrors that of a model trained directly on the

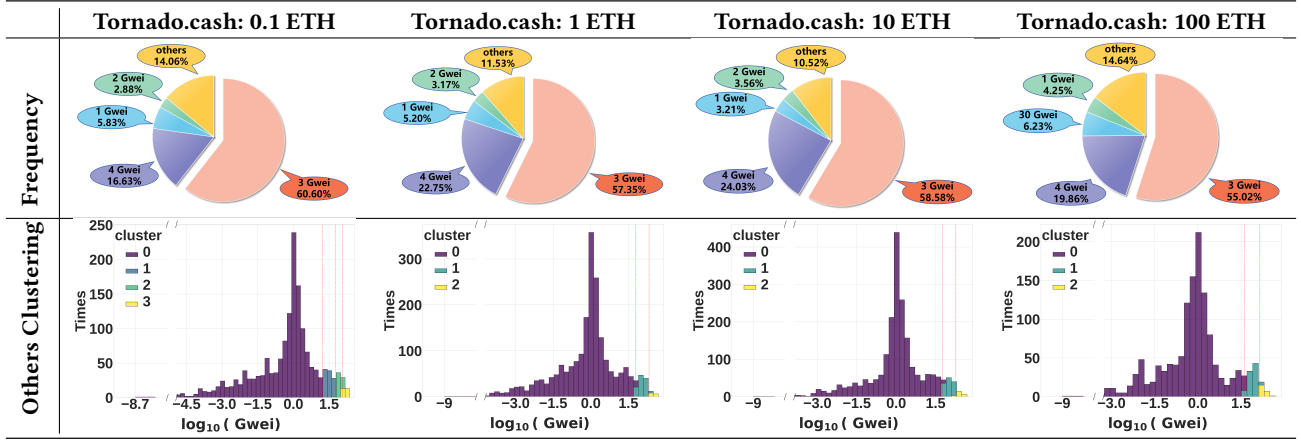
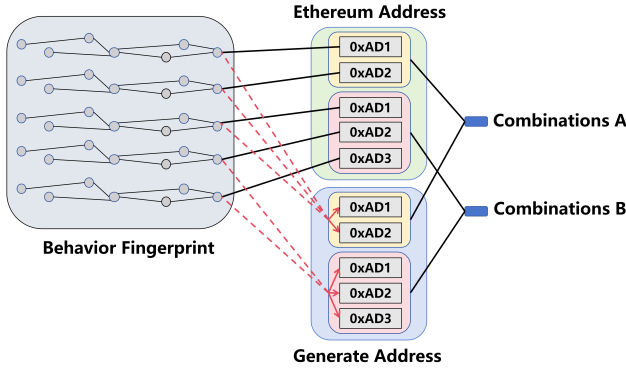
Figure 3: Probability distribution of *maxPriorityFeePerGas* across the four core Tornado.cash contracts

Figure 4: Illustration of the mapping process for behavioral fingerprints based on combinations of features

Table 5: Experimental Dataset

Dataset	Number of Addresses	Number of Transactions
<i>Dataset_S</i>	464,951	891,771
<i>Dataset_{ENS}</i>	206	206
<i>Dataset_{TC}</i>	116,180	392,167
<i>Dataset_{S1}</i>	206	206
<i>Dataset_{S2}</i>	206	206
<i>Dataset_{S3}</i>	239,172	392,167

mainnet itself. We selected MixBroker [15] and BERT4ETH [21] to serve as representative benchmarks for the supervised and unsupervised learning paradigms respectively. The specific experimental configurations are defined as follows:

MixBroker: The extremely small size of *Dataset_{ENS}* renders direct training prone to overfitting. To mitigate this issue and ensure a stable assessment, we employ ten fold cross validation.

- *E1*: 10 fold cross validation conducted on the *Dataset_{ENS}*.

Table 6: Performance Comparison of Representation Learning Methods for mixing address linking

Method		metrics				
		MAR	HR@1	@5	@10	@100
supervised	MixBroker	192	1.89%	21.31%	47.06%	50.74
	LR	206	1.53%	13.56%	48.68%	49.53%
	RF	219	1.51%	10.65%	36.88%	48.45%
	RNN	225	1.26%	8.63%	38.56%	43.86%
	LSTM	237	0.86%	7.36%	33.86%	42.38%
unsupervised	BERT4ETH	253	1.53%	15.23%	29.46%	42.35%
	Trans2Vec	391	0.99%	11.26%	18.36%	27.56%
	Diff2Vec	433	0.86%	7.31%	17.23%	25.36%
	DeepWalk	496	0.73%	6.36%	5.93%	27.09%
	Role2Vec	533	0.87%	6.53%	9.67%	23.56%

- *E2*: Training performed on *Dataset_{S1}* and testing conducted on *Dataset_{ENS}* utilizing 10 fold cross validation.

BERT4ETH:

- *E3*: Pretraining on the *Dataset_{TC}* dataset followed by testing on *Dataset_{ENS}*. The candidate set consists of addresses that performed deposit transactions prior to the withdrawal timestamp recorded in *Dataset_{ENS}*.
- *E4*: Pretraining on the *Dataset_{S3}* dataset followed by testing on *Dataset_{S3}*. The candidate set size remains identical to the one used in Experiment *E3*.

4.2.2 Results and Analysis. The experimental findings are summarized in Table 7 and Table 8. MixBroker exhibits an accuracy gap of approximately 11.9% between the *Dataset_{ENS}* and *Dataset_{S1}*, while the performance of BERT4ETH shows a rank gap of roughly 0.85% between the *Dataset_{TC}* and *Dataset_{S3}*. Detailed examination of the *Dataset_{ENS}* dataset suggests that this performance gap mainly from a significant bias regarding the activity levels of recipient. As shown in Figure 5b, the vast majority of recipient on the mainnet have never initiated a deposit transactions. Conversely,

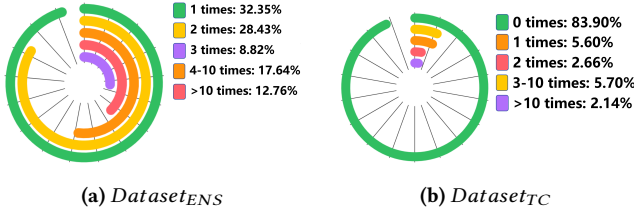


Figure 5: Distribution of deposit transaction frequencies for recipient addresses

Table 7: Validity Assessment for MixBroker

Metrics	Mainnet Data	Synthetic Data	
	E1	E2	E2*
Accuracy	0.8373	0.9367	0.8195
Precision	0.8652	0.9342	0.8489
Recall	0.8382	0.9314	0.8124
F1 Score	0.8432	0.9578	0.8214
FPR	0.1436	0.0685	0.1836
FNR	0.1618	0.1052	0.1724
AUC	0.9049	0.9781	0.8930

Table 8: Validity Assessment of BERT4ETH

Pool	Tornado.cash: 0.1 ETH			Tornado.cash: 1 ETH		
Candidates	464.5			309.1		
Metric	Rank ↓	HR@1	HR@3	Rank ↓	HR@1	HR@3
E3	200.6	1.33%	13.3%	127.9	4.76%	12.69%
E4	202.3	1.60%	12.53%	112.6	5.11%	13.10%

Figure 5a highlights that recipient in *Dataset_{ENS}* demonstrate an abnormally high frequency of deposit transactions. This discrepancy reflects the fact that *Dataset_{ENS}* consists primarily of test records shared by community members. Because these users often maintain a habit of repeatedly interacting with mixing services, the data collected through ENS fails to provide an accurate representation of authentic transaction patterns on the mainnet.

To further investigate this phenomenon, we designed an additional validation experiment *E2**: This configuration builds upon *E2* by replacing *Dataset_{S1}* with *Dataset_{S2}*. The experimental data in Table 7 demonstrate that the deviation of the accuracy metric after the replacement decreased to 2.2%. This outcome confirms that the effectiveness of the mixing address linking methods was indeed influenced by the characteristic distribution shift present in *Dataset_{ENS}*. These findings confirm that the dataset constructed in this research is effective and provides a reliable foundation for future work on mixing address linking.

4.3 Evaluation of Mixing Address Linking Methods

Using the validated *Datasets*, we conducted a comprehensive evaluation of three categories of mixing address linking methods, analyzing their performance, applicable scenarios, and limitations.

4.3.1 Heuristic Rules. Regarding high certainty heuristic rules such as same address reuse, direct transaction, and unique gas price: we argue that their success rate of linking should theoretically reach 100%, because these behaviors stem from user negligence or specific testing needs rather than random coincidence, evaluating their accuracy provides no practical value. Furthermore, the mining credit mechanism has long been discontinued and existing studies [41] record only 358 matching pairs. Consequently this evaluation focuses exclusively on the following rules: short time gap, denomination consistency, and timezone alignment.

The experimental results indicate that short time gap, denomination consistency, and timezone alignment attain accuracy levels of 0.003%, 0.024% and 0.018% respectively. These findings underscore the substantial limitations of heuristic rules based on behavior patterns within the context of mixing address linking research.

4.3.2 Representation Learning Methods. *Dataset_S* is utilized in this experiment where the candidate set size for each recipient is specified as 1000. The detailed experimental results are provided in Table 6. MixBroker demonstrates the most superior performance among all participating methods. Furthermore, the evaluation shows that supervised learning approaches generally demonstrate a performance advantage over unsupervised ones. This performance gap is primarily attributed to the ability of supervised models to leverage the label information. This observation reinforces the critical value of the high quality labeled dataset established in this study for facilitating progress in mixing address linking research.

5 Conclusion

This study presents a comprehensive methodology for constructing datasets for mixing address linking research. By simulating user interaction with mixing contracts within an Ethereum mainnet forking environment, we successfully generated an extensive-scale mixing transaction dataset without error labels. Through a series of rigorous experiments, we have validated the effectiveness of the synthetic dataset. Concurrently, we leveraged this dataset to conduct a systematic evaluation of existing mainstream mixing address linking techniques. The evaluation reveals the limitations of relying solely on heuristic rules, which show unsatisfactory accuracy in complex scenarios. In contrast, representation learning methods, particularly supervised learning models, exhibit superior association capabilities.

Acknowledgments

This research is supported by the National Key Research and Development Program of China (2023YFB2704700), the National Natural Science Foundation of China (62372485, 62332004), the Department of Education of Guangdong Province (2024ZDZX1001). Finally, we express our gratitude to Ant Group and the Guangdong Engineering Technology Research Center of Blockchain for their support.

References

- [1] 2025. Ethereum (ETH) Blockchain Explorer — etherscan.io. <https://etherscan.io/>. Accessed: Aug. 5, 2025.
- [2] 2025. foundry - Ethereum Development Framework — getfoundry.sh. <https://getfoundry.sh/>. Accessed: Aug. 5, 2025.
- [3] 2025. Lighthouse — lighthouse.one. <https://lighthouse.one/>. Accessed: Aug. 5, 2025.
- [4] 2025. Nethermind | Blockchain Research & Software Engineering — nethermind.io. <https://www.nethermind.io/>. Accessed: Aug. 5, 2025.
- [5] 2025. Rekt - Home — rekt.news. <https://rekt.news/zh/>. Accessed: Aug. 5, 2025.
- [6] 2025. (SlowMist) — cn.slowmist.com. <https://cn.slowmist.com/>. Accessed: Aug. 5, 2025.
- [7] 2025. Wallet as a Service, WEB3 Services and Consulting — blockchain-helix.com. <http://blockchain-helix.com/>. Accessed: Aug. 5, 2025.
- [8] Nesreen K Ahmed, Ryan Rossi, John Boaz Lee, Theodore L Willke, Rong Zhou, Xiangan Kong, and Hoda Eldardiry. [n. d.]. Learning role-based graph embeddings. *arXiv preprint arXiv:1802.02896* [n. d.].
- [9] Ismail Alarab, Simant Pragoonwit, and Mohamed Ikbil Nacer. 2020. Competence of Graph Convolutional Networks for Anti-Money Laundering in Bitcoin Blockchain. In *Proceedings of the 2020 5th International Conference on Machine Learning Technologies*. Association for Computing Machinery, New York, NY, USA, 23–27.
- [10] Ferenc Bérés, István András Seres, András A. Benczúr, and Mikerah Quintyne-Collins. 2021. Blockchain is Watching You: Profiling and De-anonymizing Ethereum Users. *IEEE International Conference on Decentralized Applications and Infrastructures* (2021), 69–78.
- [11] Benedikt Bünz, Shashank Agrawal, Mahdi Zamani, and Dan Boneh. 2020. Zether: Towards Privacy in a Smart Contract World. In *Financial Cryptography and Data Security: 24th International Conference, FC 2020, Kota Kinabalu, Malaysia, February 10–14, 2020 Revised Selected Papers*. Springer-Verlag, Berlin, Heidelberg, 423–443.
- [12] Vitalik Buterin et al. 2013. Ethereum white paper. In *2009 Sixth International Conference on Fuzzy Systems and Knowledge Discovery*, 5–7.
- [13] Weili Chen, Xiongfeng Guo, Zhiguang Chen, Zibin Zheng, and Yutong Lu. 2020. Phishing scam detection on Ethereum: Towards financial security for blockchain ecosystem. In *Ijcai*. Yokohama, Yokohama, Japan, 4456–4462.
- [14] Ali Dorri, Salil S Kanhere, Raja Jurdak, and Praveen Gauravaram. 2019. LSB: A Lightweight Scalable Blockchain for IoT security and anonymity. *J. Parallel and Distrib. Comput.* 134 (2019), 180–197.
- [15] Hanbiao Du, Zheng Che, Meng Shen, Liehuang Zhu, and Jiankun Hu. 2024. Breaking the Anonymity of Ethereum Mixing Services Using Graph Feature Learning. *IEEE Transactions on Information Forensics and Security* 19, 16 (2024), 616–631.
- [16] Joaquin Garcia-Alfaro, Guillermo Navarro-Arribas, Hannes Hartenstein, and Jordi Herrera-Joancomartí. 2021. Data Privacy Management, Cryptocurrencies and Blockchain Technology. In *Proceedings of the ESORICS*. Springer.
- [17] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable Feature Learning for Networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, New York, NY, USA, 855–864.
- [18] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. Curran Associates Inc., Red Hook, NY, USA, 1025–1035.
- [19] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Comput.* 9, 8 (1997), 1735–1780.
- [20] Younggeek Hong, Hyunsoo Kwon, Jihwan Lee, and Junbeom Hur. 2018. A Practical De-mixing Algorithm for Bitcoin Mixing Services. In *Proceedings of the 2nd ACM Workshop on Blockchains, Cryptocurrencies, and Contracts*. ACM, New York, NY, USA, 15–20.
- [21] Sihao Hu, Zhen Zhang, Bingqiao Luo, Shengliang Lu, Bingsheng He, and Ling Liu. 2023. Bert4eth: A pre-trained transformer for ethereum fraud detection. In *Proceedings of the ACM Web Conference 2023*. ACM, New York, NY, USA, 2189–2197.
- [22] Sijia Li, Gaopeng Gou, Chang Liu, Chengshang Hou, Zhenzhen Li, and Gang Xiong. 2022. TTAGN: Temporal transaction aggregation graph network for ethereum phishing scams detection. In *Proceedings of the ACM Web Conference 2022*. Association for Computing Machinery, New York, NY, USA, 661–669.
- [23] Dan Lin, Jiajing Wu, Qi Yuan, and Zibin Zheng. 2020. Modeling and understanding ethereum transaction records via a complex network approach. *IEEE Transactions on Circuits and Systems II: Express Briefs* 67, 11 (2020), 2737–2741.
- [24] Dan Lin, Jiajing Wu, Qi Yuan, and Zibin Zheng. 2020. T-edge: Temporal weighted multidigraph embedding for ethereum transaction network analysis. *Frontiers in Physics* 8 (2020), 204.
- [25] Xiaobo Lu, Jianbin Fang, Lin Peng, Chun Huang, Zixiao Yu, and Tiejun Li. 2025. Gator: Accelerating Graph Attention Networks by Jointly Optimizing Attention and Graph Processing. *ACM Trans. Archit. Code Optim.* 22, 2 (2025), 24 pages.
- [26] Sarah Meiklejohn and Rebekah Mercer. 2018. Möbius: Trustless tumbling for transaction privacy. *IACR Cryptol. ePrint Arch.* (2018), 881.
- [27] Satoshi Nakamoto and A Bitcoin. 2008. A peer-to-peer electronic cash system. *Bitcoin*.—URL: <https://bitcoin.org/bitcoin.pdf> 4, 2 (2008), 15.
- [28] Matthew A North. 2009. A method for implementing a statistically significant number of data classes in the Jenks algorithm. In *2009 sixth international conference on fuzzy systems and knowledge discovery*, Vol. 1. IEEE, Tianjin, China, 35–38.
- [29] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. DeepWalk: online learning of social representations. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, New York, NY, USA, 701–710.
- [30] Mario Alfonso Prado-Romero, Christian Doerr, and Andrés Gago-Alonso. 2017. Discovering bitcoin mixing using anomaly detection. In *Iberoamerican Congress on Pattern Recognition*. Springer International Publishing, Cham, 534–541.
- [31] Roman Storm. 2025. Tornado Cash Official - Secure, Decentralized, Private protocol — tornadocash-eth.ipns.dweb.link. <https://tornadocash-eth.ipns.dweb.link/>. Accessed: Aug. 5, 2025.
- [32] Benedek Rozemberczki and Rik Sarkar. 2018. Fast sequence-based embedding with diffusion graphs. In *International workshop on complex networks*, 99–107.
- [33] Jason Scharfman. 2024. The Cryptocurrency and Digital Asset Fraud Casebook, Volume II. *Springer Books* (2024).
- [34] István András Seres, Dániel A. Nagy, Chris Buckland, and Péter Burcsi. 2020. MixEth: Efficient, Trustless Coin Mixing Service for Ethereum. In *International Conference on Blockchain Economics, Security and Protocols (Tokenomics 2019)*. Schloss Dagstuhl — Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 13:1–13:20.
- [35] Omer Shlomovits and István András Seres. 2019. ShareLock: Mixing for Cryptocurrencies from Multiparty ECDSA. *Cryptology ePrint Archive*, Paper 2019/563. <https://eprint.iacr.org/2019/563>
- [36] Ardeshir Shojaeinasab, Amir Pasha Motamed, and Behnam Bahrak. 2023. Mixing detection on Bitcoin transactions using statistical patterns. *IET Blockchain* 3, 3 (2023), n/a–n/a.
- [37] Yujia Tang, Chang Xu, Can Zhang, Yan Wu, and Liehuang Zhu. 2022. Analysis of Address Linkability in Tornado Cash on Ethereum. In *China Cyber Security Annual Conference (CISC)*. Springer, Washington, DC, USA, 39–50.
- [38] Truffle Suite. 2025. Ganache - Truffle Suite — archive.trufflesuite.com. <https://archive.trufflesuite.com/ganache/>. Accessed: Aug. 5, 2025.
- [39] Shuilong Wang, Laurence T. Yang, Debin Liu, Ruonan Zhao, Xianjun Deng, Cannian Zou, and Xiaoxuan Fan. 2025. Improving Ethereum Mixing Address Linking With Tensor Computation, Neighbor Data Utilization, and Asymmetric Information Modeling. *IEEE Transactions on Information Forensics and Security* 20 (2025), 8658–8671.
- [40] Lei Wu, Yufeng Hu, Yajin Zhou, Haoyu Wang, Xiapu Luo, Zhi Wang, Fan Zhang, and Kui Ren. 2021. Towards Understanding and Demystifying Bitcoin Mixing Services. In *Proceedings of the Web Conference 2021*. Association for Computing Machinery, New York, NY, USA, 33–44.
- [41] Mike Wu, Will McTighe, Kaili Wang, István András Seres, Nick Bax, Manuel Puebla, Mariano Mendez, Federico Carrone, Tom’as De Matthey, Herman O. De-maestri, Mariano Nicolini, and Pedro Fontana. 2022. Tutela: An Open-Source Tool for Assessing User-Privacy on Ethereum and Tornado Cash. *ArXiv abs/2201.06811* (2022).
- [42] Yiding Zhang, Xiao Wang, Chuan Shi, Xunqiang Jiang, and Yanfang Ye. 2021. Hyperbolic graph attention network. *IEEE Transactions on Big Data* 8, 6 (2021), 1690–1701.

Table 9: Detailed Partitioning and Probability Distribution of Mixing Features

Category	Subtype	Value Ranges or Discrete Sets	Prob. (%)
Time (UTC)	Group 1	{3, 4, 5, 6, 11, 12, 15, 16, 17, 18, 19, 20, 21, 22}	45.57
	Group 2	{0, 2, 7, 8, 9, 10, 13, 14, 23}	46.40
	Group 3	{1}	8.03
maxPriorityFee PerGas (Gwei)	0.1 ETH	[0.65, 14.4), [14.4, 37.6), [37.6, 68.9), [68.9, 109.0]	14.60, 63.72, 7.52, 14.16
	1 ETH	{4.0}, {0.75}, [0.75, 15.5), [15.5, 36.0), [36.0, 65.0), [65.0, 134.0]	10.31, 6.53, 8.93, 51.21, 17.18, 5.84
	10 ETH	{3, 4, 19, 6, 1, 2}, [1.0, 15.4), [15.4, 44.0), [44.0, 100.0]	20.79, 14.76, 5.20, 5.20, 3.53, 0.83, 3.95, 34.10, 11.64
	100 ETH	{1, 1.8, 3, 2}, [1.0, 19.6), [19.6, 51.1), [51.1, 144.0]	6.88, 3.99, 3.62, 1.09, 28.62, 48.19, 7.61
maxFeePerGas (Gwei)	0.1 ETH	{2, 0}, [0, 11.4), [11.4, 31.52), [31.52, 98.0]	11.74, 2.35, 15.02, 48.82, 22.07
	1 ETH	{2, 0}, [0, 12.6), [12.6, 33.0), [33.02, 58.98), [58.98, 101.21]	10.31, 7.90, 8.59, 50.52, 16.49, 6.19
	10 ETH	{2.99, 2, 17.9, 5.98, 0, 3}, [0, 14.84), [43.21, 107.50]	29.67, 17.21, 7.42, 7.42, 7.12, 3.86, 12.76, 14.54
	100 ETH	{0, 2}, [0.00, 15.18), [15.18, 43.03), [111.58, 211.24]	16.00, 2.00, 26.40, 53.20, 2.40
Router	Address A	0xd90e2f925da726b50c4ed8d0fb90ad053324f31b	52.16
	Address B	0x722122df12d4e14e13ac3b6895a86e84145b6967	18.71
	Address C	0x905b63fff465b9ffbf41dea908ceb12478ec7601	14.98
	Others	Tornado.cash Direct / Personal Wallets	14.15
Relay	Personal	-	13.69
	Relayer	see Table 10	86.31
Number	Global	{1}, [2, 5], [6, 21], ≥ 22	55.63, 31.50, 11.75, 1.12
Amount	Global	{0.1, 1, 10, 100} ETH Pools	21.70, 27.78, 31.46, 19.06

6 Appendix

A Detailed Partitioning and Probability Distribution of Mixing Features

Table 9 presents the comprehensive specifications regarding value ranges and probability distributions for the seven dimensions of mixing transaction features.

Table 10: Detailed Partitioning and Probability Distribution of Relay

Relayer Address	Prob. (%)	Relayer Address	Prob. (%)
0xbe4d1e137a24af091be80ae58d652279665e3a27	5.1982%	0x8bd8746310d4ba8a0b044415bfac70db55ada5b0	0.3197%
0xa0f0287683e820ff4211e67c03cf46a87431f4e1	4.4233%	0x0e9d9a828247f5eed7f6d31d213a39805de52441	0.3192%
0xd6187b4a0f51355a36764558d39b2c21ac12393d	4.0210%	0x2faec317de7827bba0e2dcc374ce7345bd076fc7	0.3108%
0x7d3bb46c78b0c4949639ce34896bfd875b97ad08	3.8918%	0x70e688ef87f9fb0e5c738c55d4ec97333561d751	0.3083%
0x20bb3095a4852f4c97d7a188e9f7183c85acfc49	3.6167%	0xb1e20613ea9b8ad54d99e1ef38c45837f5f33271	0.2905%
0x49136693081f2c18e2cf14428dd78cd90a22dc1f	2.9878%	0x853281b7676dfb66b87e2f26c9cb9d10ce883f37	0.2855%
0x3a1d526d09b7e59fd88de4726f68a8246ddc2742	2.6503%	0x864df9cd806d58341f13602103bf853066ff962a	0.2796%
0x2d50dbcc960bec854227f51f8074e1da28bd4a37	2.4355%	0x1f2be440a818b3b87069af6ec20528c6dbec429b	0.2375%
0x155301c59ff55c74cc921da6745c86390ed13424	2.2010%	0x248b6928674b1e452d908f9aadea866a6dc4de67	0.2286%
0x03ebd2ea2b9f23669c9eb05c2a1a39f99cbdf372	2.1332%	0x47b03df2145cc9eed6d8819e02d25590f297c603	0.2281%
0x2ca1a9d6c79367ea1ea481fc0a5e8c5bd6c62d25	1.9501%	0xcdb78860218160f4b463612f30806807fe6e804c	0.2281%
0x12d92fed171f16b3a05ac1542b40648e7ced384	1.9496%	0x33350f9ffe031ca593442a189b3bc7a1ea05b3c1	0.2261%
0x62e142f218585827436f59997c301f7040396ad4	1.6409%	0x9f9f98e28456eeefc4af1c990a170e2b0d2d6027	0.2256%
0xe3e551a6a3d6664325c92833553b6bf83a865a	1.4964%	0x65214127c920b550016b18c2558c9e53e6572ecf	0.2237%
0x63606c4011e97a73bcd844cde6a38d45a728bc0e	1.4662%	0x2c36e72cf6c092f311432fca91dbc9a7fc18a115	0.2237%
0x9ffbd3f9ee795a4fda880ed553a2a4bd6d45ce5b	1.4306%	0xaaad0b504b4cd22348c4db1071736646aa314c6	0.2148%
0x4750bfc3c340aa4b31be7e71fa072716d28c29c5	1.3311%	0x85972458dfb9269567b2a27c4ffc958a4f24761	0.2133%
0x3f6ea4a39ce386505d00db7a9c91d90355b4df5d	1.3138%	0x0000208a6cc0299da631c08fe8c2ede435ea83b8	0.2024%
0xf29ff3f31ea0a4747da7421b1c4ca3620f328cf	1.3063%	0xba8d6ce9cfb1290604ebdc4fe7fc76890567b89b	0.1836%
0xc49415493eb3ec64a0f13d8aa5056f1cfc4ce35c	1.3039%	0x7e3893725d4e238b4c8c83375bbad024a66ffa42	0.1821%
0x555555731006f71f121144534ca7c8799f66aa3	1.2445%	0x52faf2849177c43595341bf7a097f27be4a5ea20	0.1801%
0xa8af5873c59dca72e632930bb683b9b749b82f3c	1.2297%	0x917d5d09cfd45b3d20d64ab6bde662dfb9d307f0	0.1737%
0x000000cd6521ed1a65fae0678ea15af4eead74fe	1.2262%	0x42fecb4137aff76e0e85702ff4f339dbf6ed859e	0.1687%
0x41a28335c5075c81502a97cebad597f28728a815	1.2138%	0x91299962ece435071129a6f19475cd701ea9eac9	0.1673%
0x0b157ac0a65319b0a18aa46b0b0d4497e1074456	1.2074%	0x4ef827a020cbcd3b0d43392cee64cd83a1db3079	0.1613%
0xdd36d585a0d32b4ae6f1790bc4292308e21a42e3	1.1124%	0x000000336020719e2ad785951582726a39fd33cf	0.1539%
0xefc225d6344d5bbb3dc9c0c3cee49b1564d25661	0.9446%	0x15980a3bd6ed317f42d2ed0dcf3d3d730b6bc0c5	0.1480%
0x0360b232285a801cbae01c984fb680ba2f465fdc	0.8645%	0xb9d8ee59a0e5f908d3a87b8b272593761b40f298	0.1425%
0x2ee39ff05643bc7cc9ed31b71e142429044a425c	0.8333%	0xd88189f7dee6e5dbd6cbc6f06fd357f4bf7f330b	0.1400%
0x36989535f0290eac96692675cbf15a3bd2f42e46	0.8086%	0xe961a2371286771258533d4155e255e501edbde2	0.1395%
0x28a51fd7b39b49ad4d7813d89e3d444d79e3ac6	0.8046%	0x7d9240d77dc24d73dd5c46e61f218fa6d2211081	0.1376%
0x6289c8a70ee2ed6914834caea431f9a82c7eaf70	0.8036%	0x30030383d959675ec884e7ec88f05ee0f186cc06	0.1356%
0x49516e20b5692839f877a7e9aa62006a5d02a7b1	0.7922%	0xffdf1dd461e3ce4a78685d2dc0641d95b71b9f53	0.1346%
0xe0f87651d3b65641c0fe1b71c5f8e8236d4b2e6	0.7729%	0x7ba6781620c91676b070d319e7e894bf4a9ec81	0.1326%
0x5c47185e0c026151c978ff1e36c1309bd71da9c3	0.7650%	0x51b75ff734670c4e9a4cc3a08ae3f8e8c993df4f	0.1282%
0xc7285e85a6d11c762a7d9c57ac38e31a671e9777	0.7408%	0xce2142247159628c02b2e672eb55492173e44a5d	0.1267%
0x0f69bea3e0aa0f40d5bce1875565ff87ab058db0	0.6067%	0x187541d7d312f742040f270d0221b4fe577934b0	0.1262%
0x08657a1f4c1f06d657f31767831421ee7fadf549	0.5888%	0xe0e484dfa7f3aa36733a915d6f07eb5a57a74a11	0.1242%
0x0f416bc2d310a1c32bc1081cf354c15fd10d4802	0.5592%	0xe0f3772995bbac19975d9fa015e3bd4167bc04e1	0.1222%
0x6a2d058890cca15beaee505caaad56b2ab54dd4	0.5493%	0xfe487df046a7e40da04a2b119d23f60987904571	0.1207%
0xa42303ee9b2ec1db7e2a86ed6c24af7e49e9e8b9	0.5418%	0xb578603d3fb9216158c29488c1a902dd0300c115	0.1202%
0x80e179c4a2f057be91573b32bece53888fb48311	0.5404%	0xab1723831396351f759d0c8787e5ddaa18f5424c	0.1197%
0xd04e9f0945dea8373d882c730e2c93a74b591796	0.5310%	0x82e6b31b0fe94925b9cd1473d05894c86f277398	0.1197%
0x3e9979106da74afb64b866218aeb47f224a312bb	0.4998%	0x74b6ea6b2eefd3ef4da5e8c4c0480776035029c2	0.1163%
0xefab18983029d2ba840e34698efb67fdf8120711	0.4928%	0xaf301de836c81deb8dff9dc22745e23c476155b2	0.1108%
0xf18673ab6eb72937607aa8388b8f7aa0ac3a0d32	0.4493%	0xfa1c9d758af489a1055586de4d5f42ec2db91522	0.1099%
0x1247749d7e28d357b4279110af0802603ac526ce	0.4142%	0x1f18a352d143e9330ab6a551d620d3b86236af71	0.1099%
0x78c88ff43cd503316e8a15b6d92b2ebfa73802b2	0.4127%	0xb326d1f0837e14ad265397800ef3bf7a538335e4	0.1094%
0x03392600086874456e086d2bac104380bcdebcfc0	0.3884%	0x13bb354793913f9e5dcfa52f12236e02a6438da	0.1074%
0x9ee26a4bfd731e8e742b65bf955814eadd7f151	0.3721%	0x3884e9b1e2b0f8d00666c9767b5602b709eeee06	0.1049%
0x81c764add7a228ccd3b7fad1f46e06d55c39f7a2	0.3642%	0x8d10181aa84ea1e809bd6c838104d6bc932fcd1e	0.1019%
0x3de77ca446bbe9370ea86814beaea9de119398cb	0.3424%	0x40f488e7f284f74251d622fd75a717579c369bd0	0.1005%
0x1036af02bcdb2e3a4db2d3d40b29e5054edc79ba	0.3281%	0x87bedf6ad81a2907633ab68d02c44f0415bc68c1	0.1000%

B Detailed Partitioning and Probability Distribution of Relay

Table 10 enumerates the specific relay addresses and their corresponding selection probabilities utilized within the transaction generation process.